

# Sicurezza nelle applicazioni, parte 2

Corso di Sicurezza e affidabilità

Anno accademico 2024-2025

# Argomenti

- Schemi d'attacco basati su buffer manipulation:
  - Buffer overflow
  - Buffer overread; vulnerabilità CVE-2014-0160 (Heartbleed)
- Schemi d'attacco basati su null pointer dereference
  - Vulnerabilità CVE-2009-2692
  - Vulnerabilità CVE-2009-1897
- Java secure coding guidelines

Schemi d'attacco basati su buffer  
manipulation

# I buffer

- Con il termine buffer si intende uno spazio di memoria nel quale l'applicazione conserva una certa quantità di dati
- Tipicamente viene implementato con un array di un determinato tipo di dati primitivo (ad esempio byte) o con una stringa mutabile
- In alcuni linguaggi di programmazione, e specialmente nel linguaggio C, non viene effettuato alcun controllo che la lettura o la scrittura in un buffer non esca dai limiti del buffer stesso
- Si parla di:
  - Buffer overflow: scrittura fuori dai limiti di un buffer
  - Buffer overread: lettura fuori dai limiti di un buffer

## Secondo la tassonomia...

- CWE-664: Improper Control of a Resource Through its Lifetime
  - CWE-118: Incorrect Access of Indexable Resource ('Range Error')
    - CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer
- CWE-707: Improper neutralization
  - CWE-240: Improper handling of inconsistent structural elements
    - CWE-130: Improper handling of length parameter inconsistency
- CAPEC-255: Manipulate data structures
  - CAPEC-123: Buffer manipulation
    - **CAPEC-100: Overflow buffers**
    - **CAPEC-540: Overread buffers**

# Le weakness coinvolte

- Diverse weakness possono essere sfruttata per causare una lettura/scrittura fuori dai limiti di un buffer
- La weakness più semplice (CWE-119) si presenta quando il programma, sotto certe condizioni, effettua un accesso al buffer con un indice al di fuori del range del buffer stesso
- Un'ulteriore weakness (CWE-130) consiste nel non gestire un valore inconsistente di un parametro di lunghezza: ossia, un parametro che dovrebbe indicare la dimensione di un certo insieme di dati di input, ma che ha un valore diverso dalla effettiva dimensione dei dati
- Quest'ultima weakness può generare una lettura/scrittura fuori dai limiti del buffer quando i dati sono trasferiti nel/letti dal buffer

# Vulnerabilità con gets

```
...  
char buf[BUFSIZE];  
gets(buf);  
...
```

- La funzione di libreria C `gets()` legge l'input da tastiera e lo trasferisce in un buffer, ma non verifica che l'input possa essere contenuto nel buffer
- Nei moderni compilatori questa funzione è deprecata proprio perché porta automaticamente a weakness del tipo CWE-119
- Al suo posto dovrebbe essere invocata `fgets(buf, max, file)`
  - Copia al più max bytes da file a buf
  - La copia termina quando viene incontrato un carattere "a capo"
- Tuttavia è sufficiente fornire un valore errato di max (maggiore della dimensione del buffer) per introdurre una vulnerabilità

# Vulnerabilità con fgets

```
void readFile(char *fileName) {  
    char buf[256];  
    int bytes = 300;  
    FILE* file;  
    file = fopen(fileName,"r");  
    fgets(buf, bytes, file);  
    printf("Contenuto: %s\n",buf);  
}  
  
int main(int argc, char *argv[]) {  
    readFile(argv[1]);  
    printf("Premi invio per terminare.");  
    getc(stdin);  
    return 0;  
}
```

- In una versione precedente del programma il buffer era di 300 bytes
- Successivamente il buffer è stato ristretto a 256 bytes ma il programmatore non ha aggiornato la variabile bytes
- Ecco perché bisogna sempre utilizzare le costanti simboliche, e non i numeri magici...

# Una variante

```
void readFile(char *fileName) {  
    char buf[256];  
    int bytes;  
    FILE* file;  
    file = fopen(fileName,"r");  
    printf("Bytes to copy:\n");  
    scanf("%d", &bytes);  
    fgets(buf, bytes, file);  
    printf("Contenuto: %s\n",buf);  
}  
  
int main(int argc, char *argv[]) {  
    readFile(argv[1]);  
    printf("Premi invio per terminare.");  
    getc(stdin);  
    return 0;  
}
```

- In questo caso il programma ottiene la dimensione dell'input dall'utente
- La funzione fgets viene usata assumendo che la dimensione data dall'utente sia inferiore rispetto alla dimensione del buffer
- Questo è un esempio di weakness CWE-130 (improper handling of length parameter inconsistency)

Buffer overflow

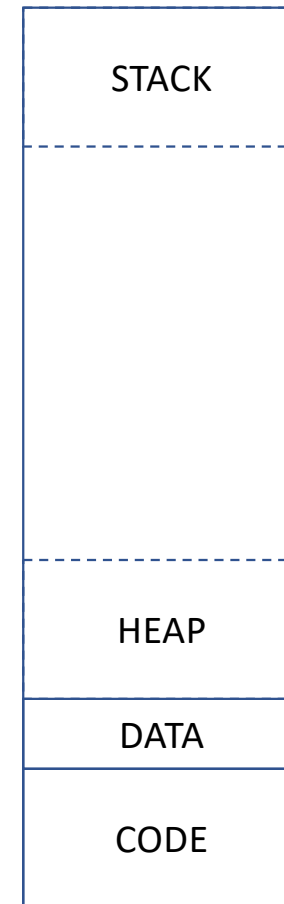
# Conseguenze del buffer overflow

- Quando si ha un buffer overflow, viene sovrascritta l'area di memoria adiacente al buffer
- Nel linguaggio C, ciò può essere particolarmente interessante per un attaccante quando il buffer si trova tra le variabili locali di un programma
- Come sono organizzate le variabili locali?
- (N.B. supporremo una architettura hardware a 32 bit)

# La memoria di un processo C

```
int f(int i) {  
    int pass;  
    int buff[8];  
    ...  
}
```

```
int main(int argc, char* argv[]) {  
    int x;  
    ...  
    f(8);  
    ...  
}
```



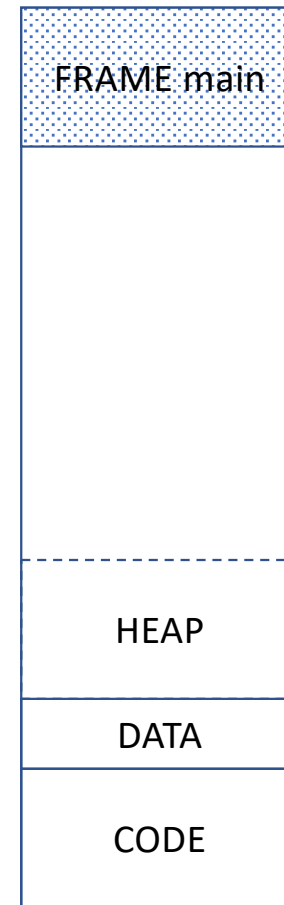
# La memoria di un processo C

```
int f(int i) {  
    int pass;  
    int buff[8];  
    ...  
}
```

```
➡ int main(int argc, char* argv[]) {  
    int x;  
    ...  
    f(8);  
    ...  
}
```

Base pointer →

Stack pointer →



# La memoria di un processo C

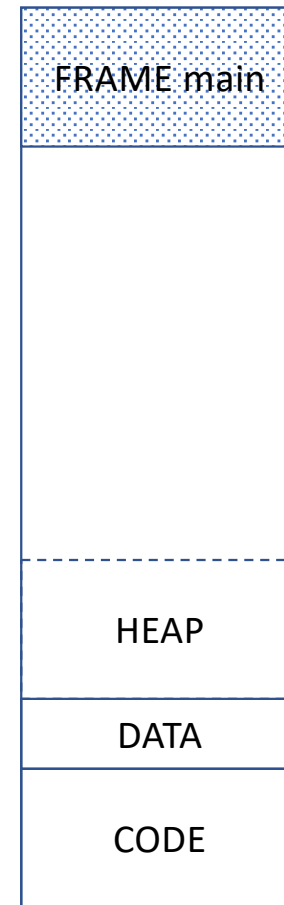
```
int f(int i) {  
    int pass;  
    int buff[8];  
    ...  
}
```

```
int main(int argc, char* argv[]) {  
    int x;  
    ...  
    f(8);  
    ...  
}
```



Base pointer →

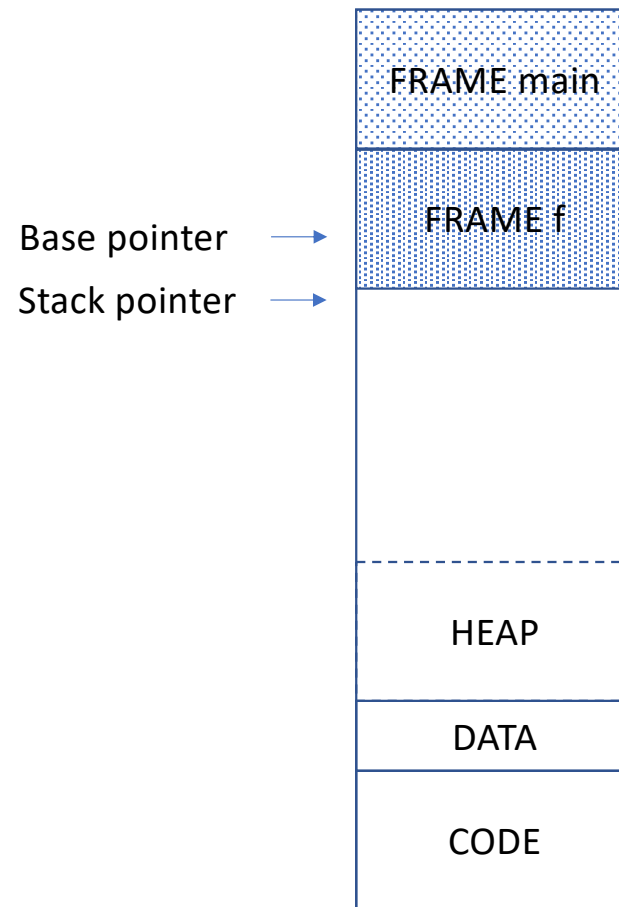
Stack pointer →



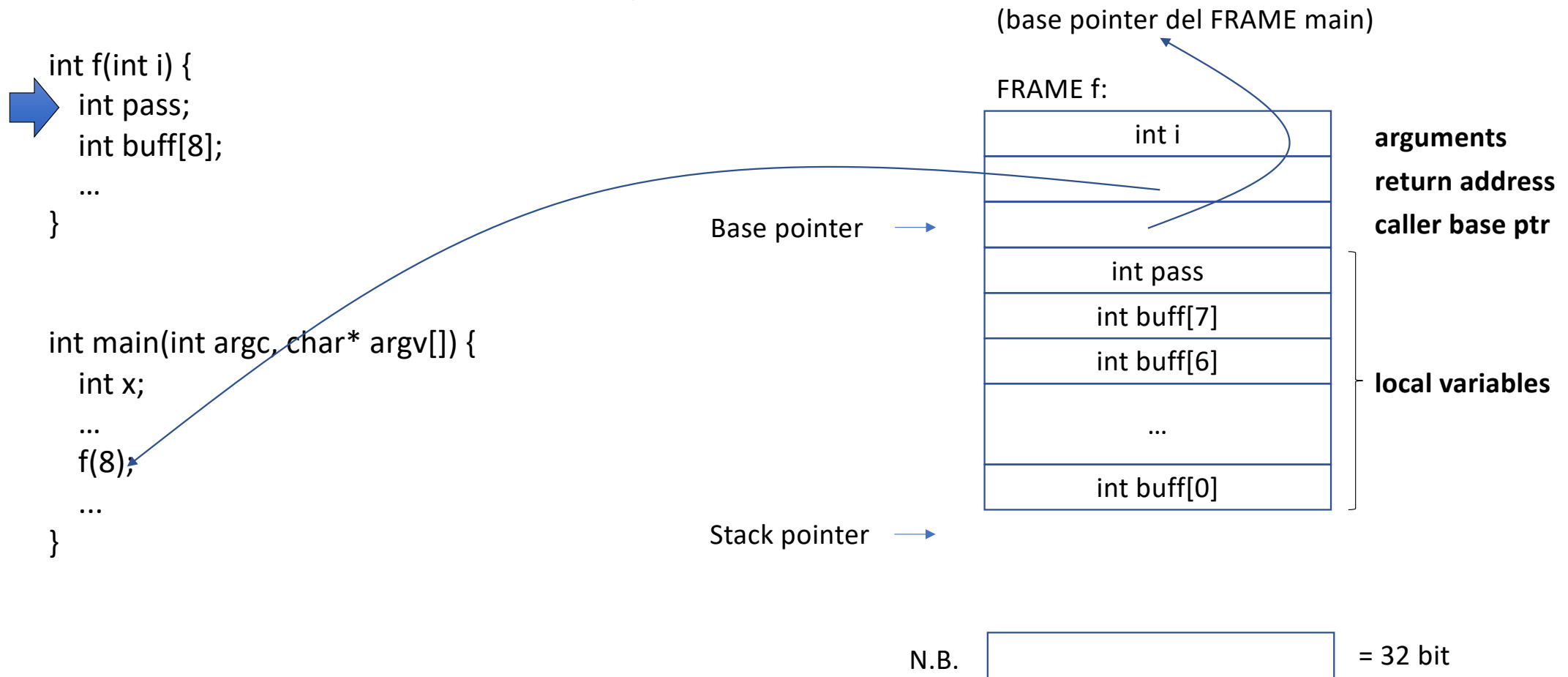
# La memoria di un processo C

```
➡ int f(int i) {  
  int pass;  
  int buff[8];  
  ...  
}
```

```
int main(int argc, char* argv[]) {  
  int x;  
  ...  
  f(8);  
  ...  
}
```



# La memoria di un processo C



# Cosa stampa questo programma?

```
int f(int i) {  
    int pass;  
    int buff[8];  
    pass=0;  
  
    printf("%d\n", pass);  
  
    return 0;  
}
```

|   |
|---|
| 0 |
| 1 |

```
int main(int argc, char* argv[]) {  
    int x;  
    x=1;  
    f(8);  
    printf("%d\n",x);  
}
```

# Cosa stampa quest'altro programma?

```
int f(int i) {  
    int pass;  
    int buff[8];  
    pass=0;  
    buff[i]=97;  
    printf("%d\n", pass);  
  
    return 0;  
}
```

|    |
|----|
| 97 |
| 1  |

```
int main(int argc, char* argv[]) {  
    int x;  
    x=1;  
    f(8);  
    printf("%d\n",x);  
}
```

|                                                    |
|----------------------------------------------------|
| Abbiamo modificato il valore della variabile pass! |
|----------------------------------------------------|

# Come mai?

- Questo è un esempio di scrittura out-of-bound
- L'array buff infatti ha dimensione 8, e quindi i suoi indici corretti variano da 0 a 7
- L'istruzione segnata in rosso, invece, effettua un assegnamento `buff[8] = 97`
- Che significato ha?

# Accesso ad array in C

- In C la scrittura `buff[i]` viene interpretata in questo modo:
  - Prendi l'indirizzo di memoria della base dell'array (`buff[0]`), ad esempio `0xffff fbe0`
  - Moltiplica `i` per la dimensione (in byte) di un elemento dell'array
  - Somma tale valore all'indirizzo della base dell'array
  - Questo è l'indirizzo dell'elemento dell'array da leggere/scrivere
- Esempio: `buff[1] == 0xffff fbe0 + (1 * 4) = 0xffff fbe4`
- Ma `buff[8]` è la posizione nel frame della parola di 32 bit subito «sopra» al buffer
- In tale posizione c'è `pass`!

**`buff[i] = 97;`**

|             |                 |                     |
|-------------|-----------------|---------------------|
| 0xffff fc0c | 8               | i                   |
| 0xffff fc08 | ...             | return address      |
| 0xffff fc04 | ...             | caller BP           |
| 0xffff fc00 | <del>0</del> 97 | pass <b>buff[8]</b> |
| 0xffff fbfc | ?               | buff[7]             |
| 0xffff fb8  | ?               | buff[6]             |
|             | ...             |                     |
| 0xffff fbe4 | ?               | buff[1]             |
| 0xffff fbe0 | ?               | buff[0]             |

## E ancora...

- Allo stesso modo possiamo sovrascrivere altre regioni del frame di attivazione
- Ad esempio, `buff[10]` corrisponde alla regione che contiene l'indirizzo di ritorno dall'invocazione di `f`
- Se sovrascriviamo tale regione possiamo far sì che l'invocazione di `f`, anziché ritornare al `main`, termini saltando ad una funzione da noi scelta

**`buff[i] = &badCode;`**

|             |                                    |                           |
|-------------|------------------------------------|---------------------------|
| 0xffff fc0c | 10                                 | i                         |
| 0xffff fc08 | <del>...</del> <b>&amp;badCode</b> | return a. <b>buff[10]</b> |
| 0xffff fc04 | ...                                | caller BP <b>buff[9]</b>  |
| 0xffff fc00 | 0                                  | pass <b>buff[8]</b>       |
| 0xffff fbfc | ?                                  | buff[7]                   |
| 0xffff fbfc | ?                                  | buff[6]                   |
|             | ...                                |                           |
| 0xffff fbe4 | ?                                  | buff[1]                   |
| 0xffff fbe0 | ?                                  | buff[0]                   |

# I precedenti esempi nella VM del corso

- Login = user, password = user
- Gli esempi si trovano in  
/home/user/Workspaces/workspaceBufferOverflow/VarOverwrite
- Esegui i seguenti comandi dopo aver visionato il codice dei file .c:

```
gcc calls_v1.c  
./a.out
```

```
gcc calls_v2.c  
./a.out
```

```
gcc calls_v2_extended.c (stampa memoria e registri)  
./a.out
```

# Attacchi buffer overflow

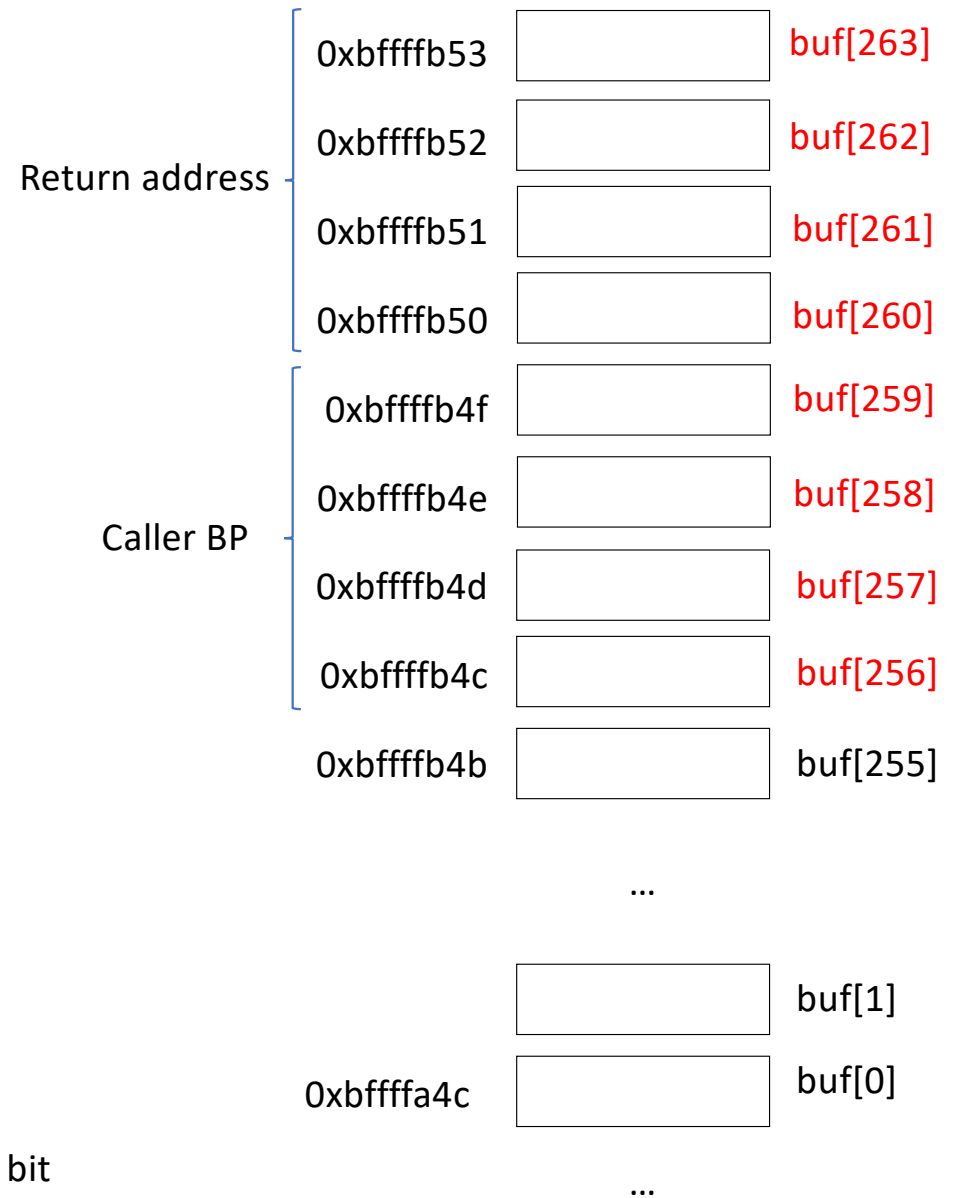
- Di regola l'attacco viene condotto verso applicazioni vulnerabili che godono di particolari privilegi, ad esempio di superuser
- L'attaccante riempie un buffer con codice malevolo, e quindi sfrutta l'overflow per sovrascrivere l'indirizzo di ritorno del frame corrente dell'applicazione attaccata, in maniera che passi ad eseguire il codice malevolo
- Il codice malevolo è spesso il cosiddetto «shellcode», ossia una chiamata di sistema `execve` che sostituisce il programma corrente con una shell
- In tal modo l'attaccante ottiene una shell con i privilegi di superuser!

# Esempio di attacco

```
void readFile(char *fileName) {  
    char buf[256];  
    int bytes;  
    FILE* file;  
    file = fopen(fileName,"r");  
    printf("Bytes to copy:\n");  
    scanf("%d", &bytes);  
    fgets(buf, bytes, file);  
    printf("Contenuto: %s\n",buf);  
}
```

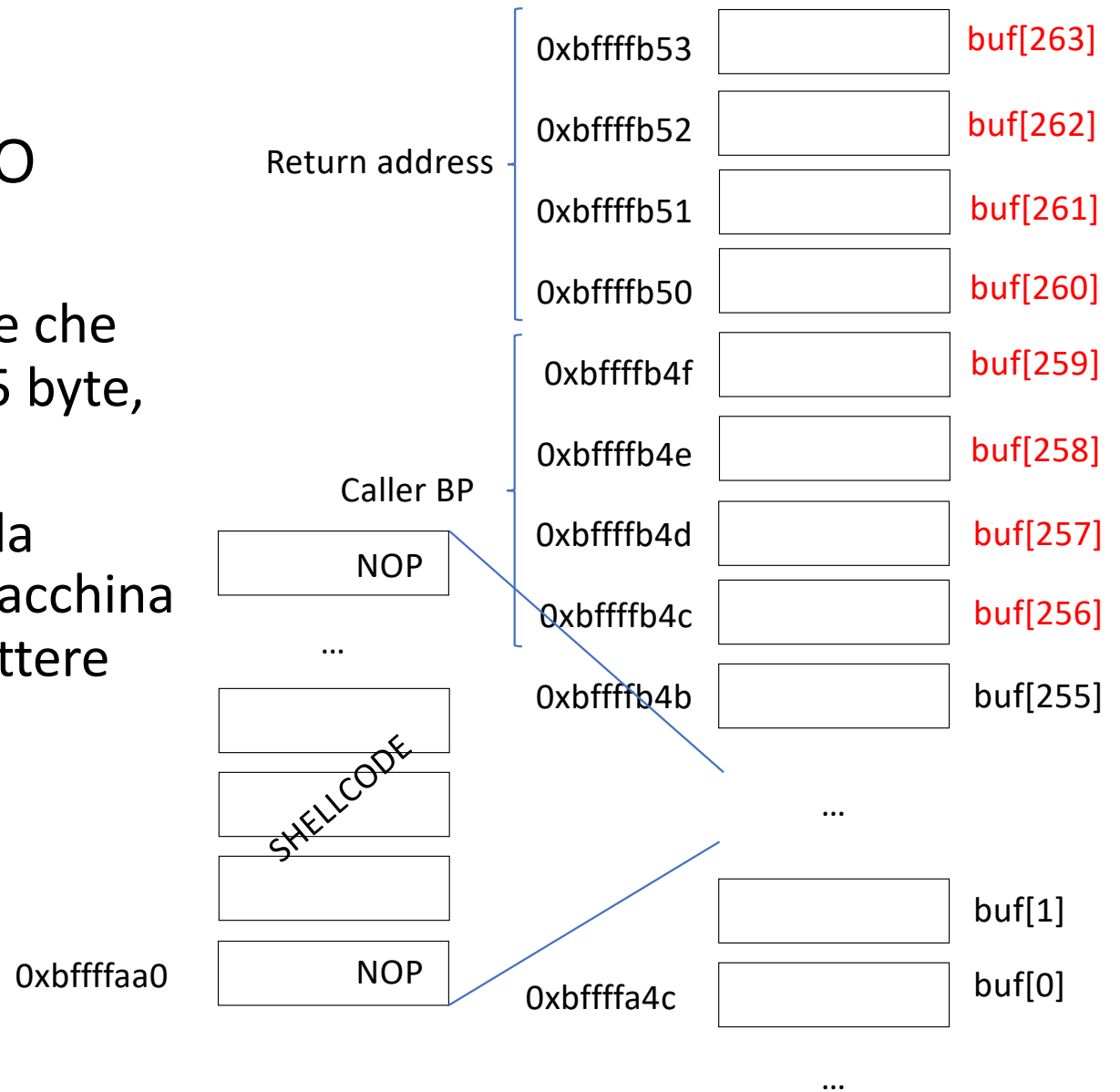
```
int main(int argc, char *argv[]) {  
    readFile(argv[1]);  
    printf("Premi invio per terminare.");  
    getc(stdin);  
    return 0;  
}
```

N.B.  = 8 bit



# Esempio di attacco

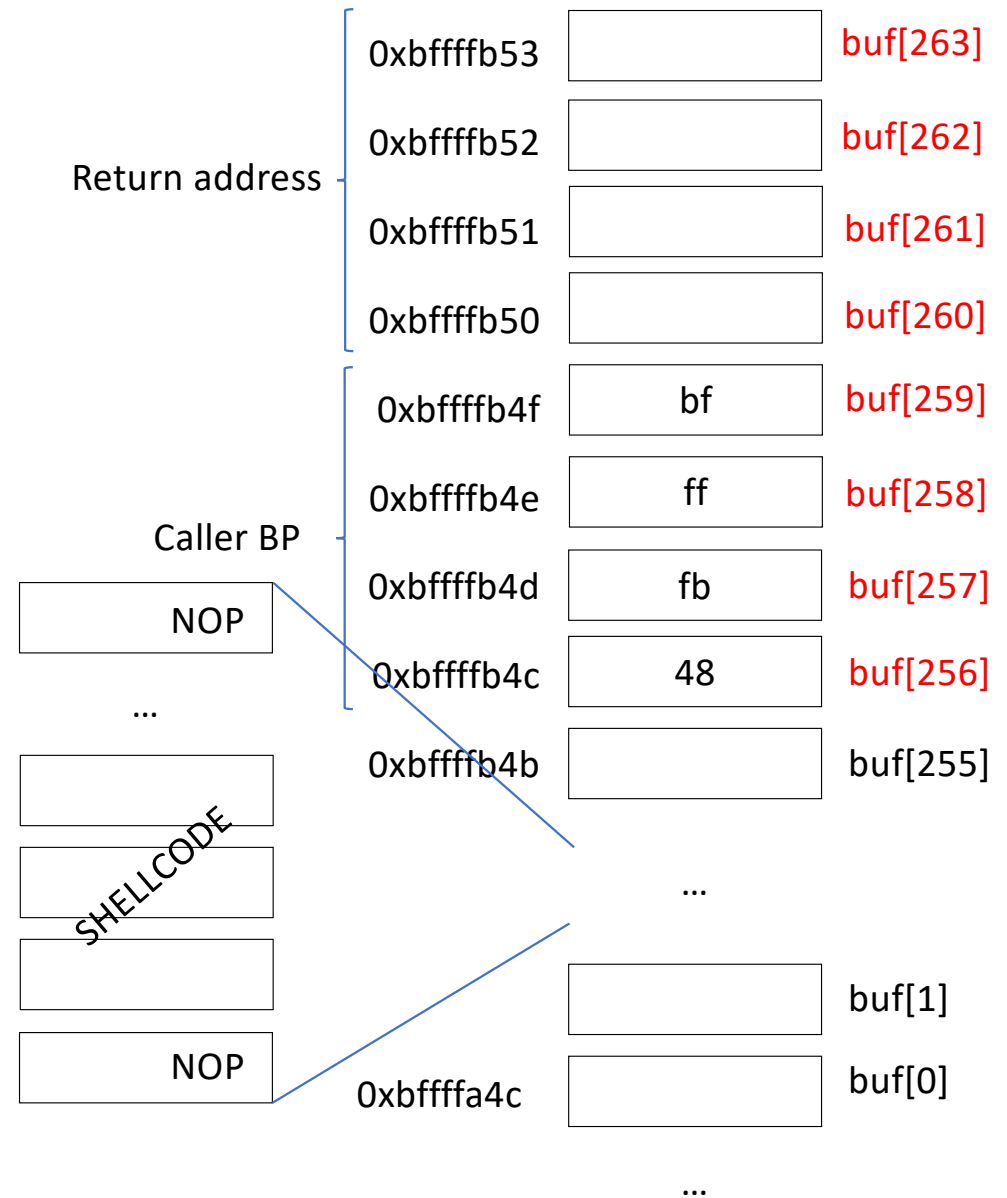
- L'attaccante prepara un file che contiene, entro i primi 255 byte, lo shellcode
- Lo shellcode viene di regola circondato da istruzioni macchina nop, in maniera da permettere anche salti imprecisi



# Esempio di attacco

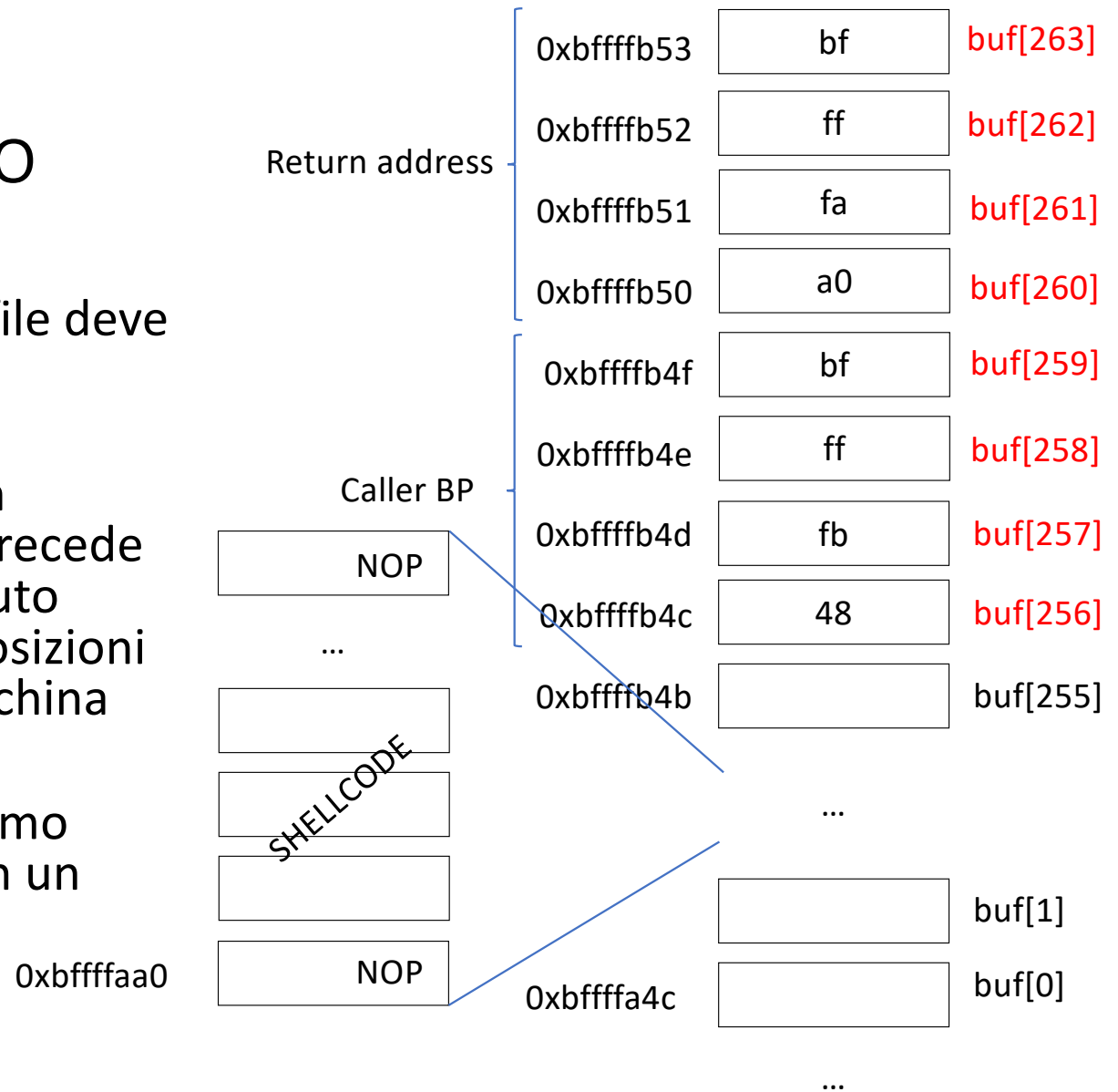
- I byte 256-259 devono contenere un base pointer corretto, altrimenti viene generato un segmentation fault quando la funzione termina
- Si può ad esempio utilizzare il BP del chiamante; tale valore può essere individuato monitorando con un debugger un'esecuzione precedente del sistema (gli indirizzi di memoria sono logici e non variano di esecuzione in esecuzione)
- Un altro approccio consiste nell'eseguire più volte l'attacco con valori casuali, finché non si osserva che l'attacco va a buon fine

0xbfffaa0

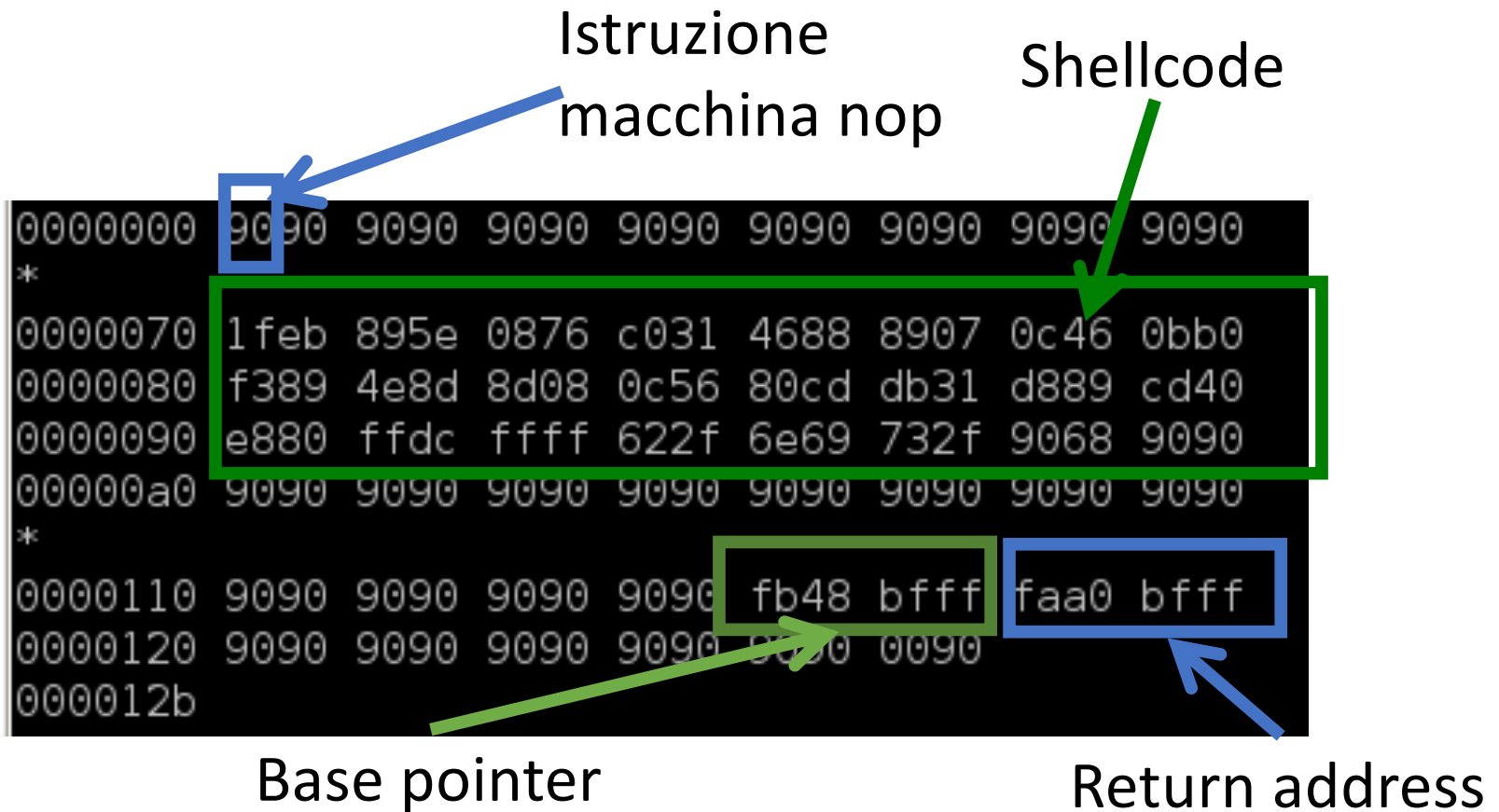


# Esempio di attacco

- Infine nei byte 260-263 del file deve venir posto l'indirizzo dello shellcode
- Questo può essere anche un indirizzo interno a buf che precede lo shellcode, se abbiamo avuto l'accortezza di riempire le posizioni precedenti di istruzioni macchina nop
- Anche in questo caso possiamo individuare tale indirizzo con un debugger



# Il file d'attacco



# Il precedente esempio nella VM del corso

- Occorre disabilitare le protezioni:
  - Riavviare il sistema selezionando il kernel NO\_EXEC
  - Da una shell disabilitate la randomizzazione dello spazio di memoria:

```
su (password: root)
echo 0 > /proc/sys/kernel/randomize_va_space
exit
```

- Gli esempi si trovano in /home/user/Workspaces/workspaceBufferOverflow/FunctionCall
- Eseguire i seguenti comandi:

```
make leggiFile
./leggiFile file0k.txt
```

(il programma stampa "Contenuto:" e il contenuto del file)

(leggere il programma createExploitFile.c e seguire le istruzioni per creare l'exploit)

```
make createExploitFile
./createExploitFile
hexdump exploit.txt
./leggiFile exploit.txt
```

(crea il file exploit.txt)  
(il contenuto del file è quello riportato nella slide precedente)  
(viene aperta una nuova shell)

# Heap buffer overflow

- Abbiamo fino ad ora mostrato esempi di buffer overflow che colpiscono buffer allocate sullo stack
- Un buffer overflow può riguardare anche buffer allocati sullo heap
- Questi overflow non possono essere sfruttati per riscrivere il return address
- Ma si possono riscrivere altri dati, ad esempio puntatori a funzione (che, come vedremo, sono molto utilizzati nei driver delle periferiche)

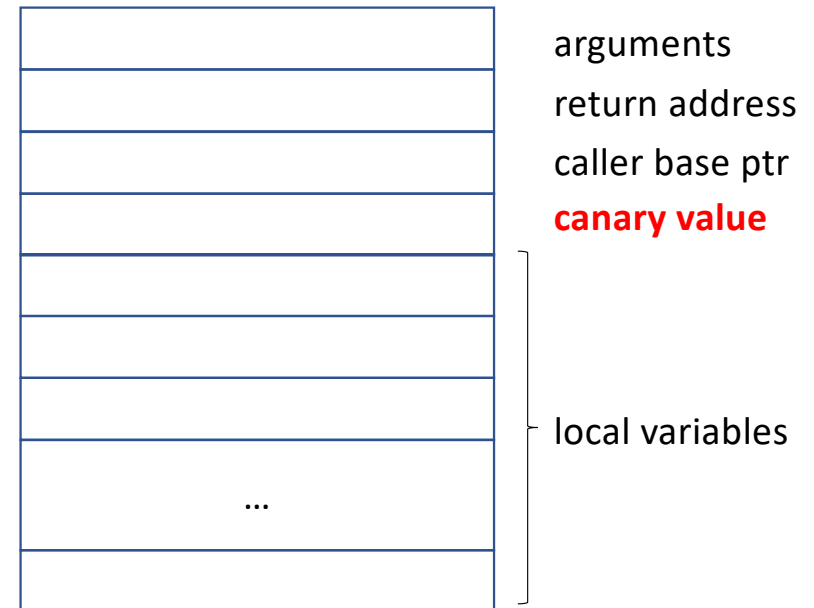
# Contromisure

- Canary values
- Address space layout randomization (ASLR)
- Executable space protection

# Canary values

- Istruzioni inserite dal compilatore a compile time
- Ad ogni invocazione di funzione viene inserito un valore di controllo (canary value) sullo stack, tra BP e variabili locali
- Il canary value è tipicamente un valore random, generato in fase di inizializzazione del programma e salvato in un'area specifica di memoria, "lontana" dallo stack
- Il canary value viene controllato all'uscita dalla funzione e deve essere identico al valore inserito in precedenza
- Se il controllo fallisce il programma viene terminato con un messaggio di errore
- Il programma non deve contenere weakness che permettono letture out-of-bound (buffer overread), o l'attaccante potrebbe essere in grado di copiare il canary value

FRAME f:



# Opzioni di protezione fornite dal compilatore GCC

- **-fstack-protector** (default per 4.8.3)
  - Genera codice aggiuntivo per verificare la presenza di buffer overflow.
  - Aggiunge un canary value (guard variable) solo alle:
    - Funzioni che invocano la funzione di sistema “alloca” (allocazione dinamica su stack)
    - Funzioni con buffers più grandi di 8 bytes.
- **-fstack-protector-all**
  - Protegge tutte le funzioni
  - Molto più costoso
- **-fstack-protector-strong** (gcc 4.9)
  - Compromesso tra completezza e overhead
  - Attivato in quelle funzioni per cui:
    - L'indirizzo di una variabile locale viene usato nella parte destra di un assegnamento, o come parametro passato a funzione
    - Dichiarare variabili locali di tipo array
    - Usa “register local variables”

# Address Space Layout Randomization (ASLR)

- Contromisura implementata nei moderni sistemi operativi
  - Alcuni indirizzi di memoria del virtual address space del processo vengono modificati da un'esecuzione ad un'altra
  - Rende più difficile la scrittura degli exploit, ad esempio diventa difficile "indovinare" l'indirizzo in memoria del buffer, e quindi l'indirizzo da usare per riscrivere il return address o il BP
- In Linux eseguire i seguenti comandi:
  - Per disabilitarla  
`$ echo 0 > /proc/sys/kernel/randomize_va_space`
  - Per randomizzare l'indirizzo di stack, virtual dynamic shared object (VDSO) page, e shared memory regions  
`$ echo 1 > /proc/sys/kernel/randomize_va_space`
  - Per randomizzare anche il segmento dati:  
`$ echo 2 > /proc/sys/kernel/randomize_va_space`

# Executable Space Protection (1)

- Termine che indica la possibilità per il sistema operativo di distinguere aree di memoria contenente codice da aree di memoria contenente dati
- Spesso richiede il supporto sia da parte del sistema operativo che da parte dell'hardware
- Nei processori con MMU un opportuno bit di protezione della tabella delle pagine indica se una pagina è eseguibile o meno
  - Intel : XD bit eXecute Disable.
  - AMD: Enhanced Virus Protection.
  - ARM v6+: XN (eXecute Never)

# Executable Space Protection (2)

- Nei sistemi operativi il supporto all'Executable Space Protection è stato introdotto dopo il 2000
- Linux: dal 2004 con NX bit
  - Per disabilitarla occorre editare il file `/boot/grub/menu.lst` ed aggiungere le opzioni `noexec=off noexec32=off`
  - Nel caso in cui l'hardware non la supporti è possibile simularla attraverso le patch ExecShield e PAX (non abilitate di default)
- Windows: dal 2004 (Windows XP) con NX bit, detta Data Execution Prevention (DEP).
  - Simulazione non possibile, è solo possibile prevenire attacchi che sfruttano Shell Exception Handler
  - Con Windows Vista e Windows Server 2008 anche ASLR

# Esempi di esecuzione protetta con la VM del corso

- Usare gli esempi in  
/home/user/Workspaces/workspaceBufferOverflow/FunctionCall
- Eseguire:  
\$ make safe (compila con -f-stack-protector-all)  
\$ ./leggiFile exploit.txt
- Osservare il messaggio di errore
- Abilitare randomize\_va\_space ed eseguire  
\$ rm leggiFile  
\$ make (compila senza -f-stack-protector-all)  
\$ ./leggiFile exploit.txt
- Osservare che lo shellcode non viene eseguito perchè la randomizzazione dell'indirizzo ha fatto puntare ad un area di memoria non valida

# JIT spraying

- Il runtime di alcuni linguaggi (Java, .NET) fa uso di compilatori just-in-time (JIT) che generano al volo codice eseguibile in linguaggio macchina
- Inoltre tali runtime sono complessi, e quindi facilmente contengono vulnerabilità
- Il codice macchina generato dal compilatore JIT deve essere necessariamente posizionato su pagine eseguibili
- Alcuni attacchi (detti JIT spraying) sfruttano quindi la presenza del compilatore JIT per aggirare l'executable space protection:
  - L'attaccante usa il compilatore JIT per generare del codice malevolo, ad esempio a partire da script Javascript
  - Quindi sfrutta buffer overflow (o use after free) per redirezionare l'esecuzione verso il nuovo codice

# Buffer overflows nei linguaggi managed

- Gli array nei linguaggi managed (Java, .NET, Python) sono protetti da buffer overflow
- Tuttavia il codice nativo che interagisce con un programma managed può essere affetto da vulnerabilità di questo tipo:
  - Le macchine virtuali (JVM, CLR) sono di regola scritte in C/C++ e quindi possono esse stesse contenere vulnerabilità
  - Un programma managed può invocare librerie implementate nativamente, che potrebbero essere vulnerabili
  - Il compilatore JIT o l'interprete del linguaggio possono presentare difetti che causano l'assenza di controlli su array

# Riferimenti aggiuntivi

- <http://phrack.org/issues/49/14.html#article> - Smashing the stack for fun and profit
- <http://www.mgraziano.info/docs/stsi2010.pdf> - Smashing the stack in 2010
- <http://paulmakowski.wordpress.com/2011/01/25/smashing-the-stack-in-2011/> - Smashing the stack in 2011
- <http://www.outflux.net/blog/archives/2014/01/27/fstack-protector-strong/> - on -fstack-protector-strong

Buffer overread

# Buffer overread

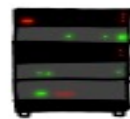
- Con buffer overread si intende un attacco il quale determina la lettura di una zona di memoria al di là dei limiti di un determinato buffer
- Tale zona di memoria potrebbe contenere informazioni sensibili, che potrebbero essere rivelate all'attaccante
- Un esempio è l'attacco che sfrutta la vulnerabilità CVE-2014-0160 (Heartbleed)

## HOW THE HEARTBLEED BUG WORKS:

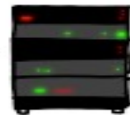
SERVER, ARE YOU STILL THERE?  
IF SO, REPLY "POTATO" (6 LETTERS).



...this pages about "boats". User Erica Hayes  
secure connection using key "4538538374224".  
User Meg wants these 6 letters: POTATO. User  
Ada wants pages about "irl games". Unlocking  
secure records with master key 5130985733435  
terrie (chrome user) sends this message: "H



POTATO

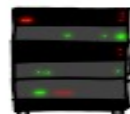


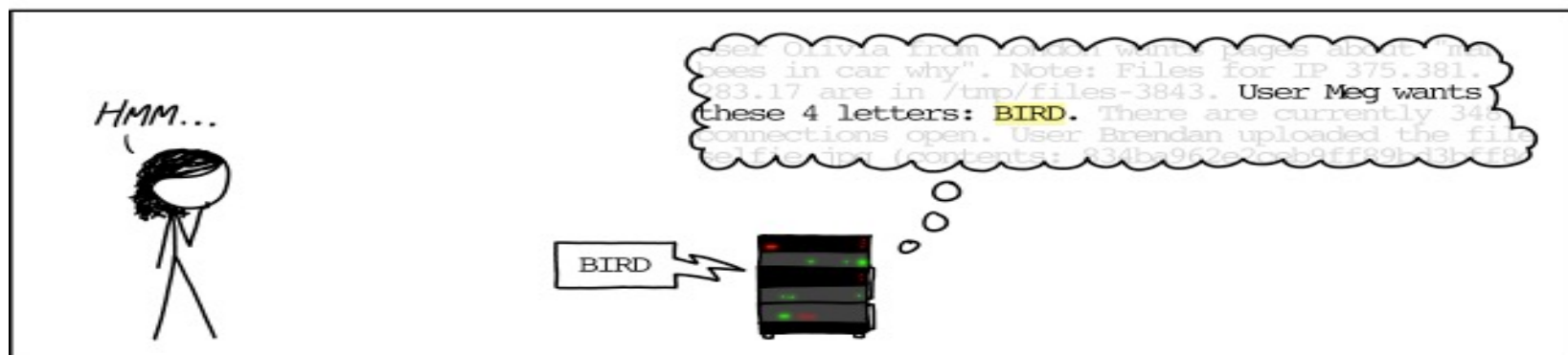
...this pages about "boats". User Erica Hayes  
secure connection using key "4538538374224".  
User Meg wants these 6 letters: **POTATO**. User  
Ada wants pages about "irl games". Unlocking  
secure records with master key 5130985733435  
terrie (chrome user) sends this message: "H

SERVER, ARE YOU STILL THERE?  
IF SO, REPLY "BIRD" (4 LETTERS).

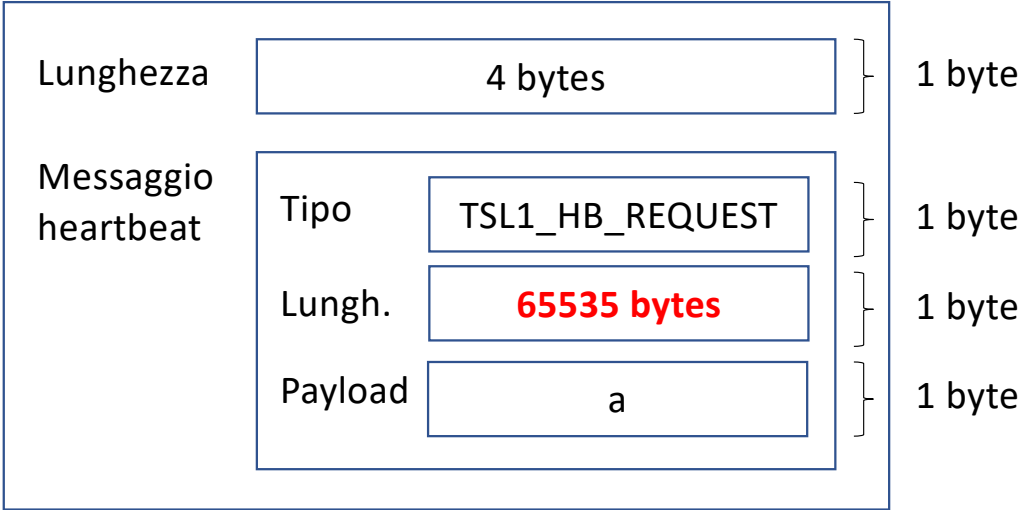


User Olivia from London wants pages about "m  
bees in car why". Note: Files for IP 375.381.  
383.17 are in /tmp/files-3843. User Meg wants  
these 4 letters: BIRD. There are currently 346  
connections open. User Brendan uploaded the file  
elfe/ing (contents: 834ba962e2ceb9ff89b33bf8



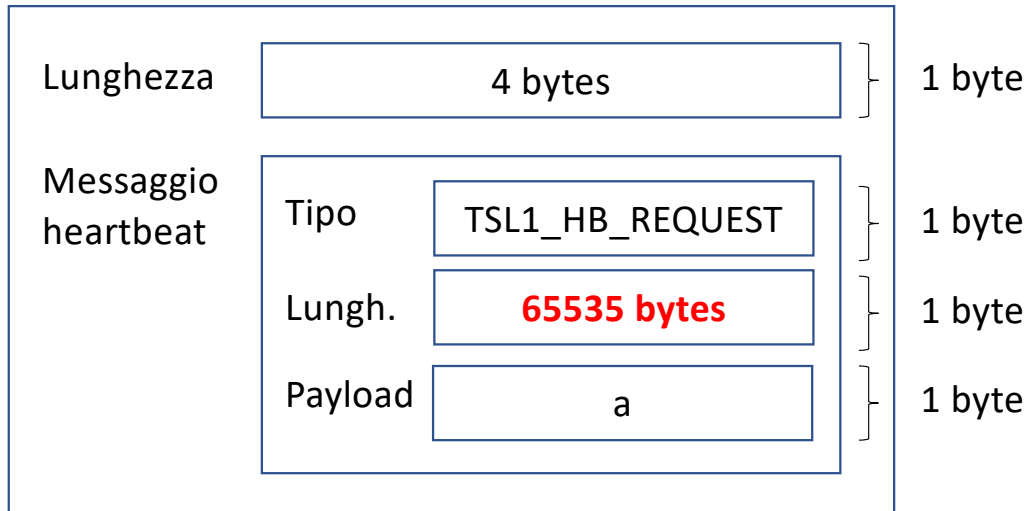


Record SSLv3



Heartbeat inviato alla vittima

## Record SSLv3

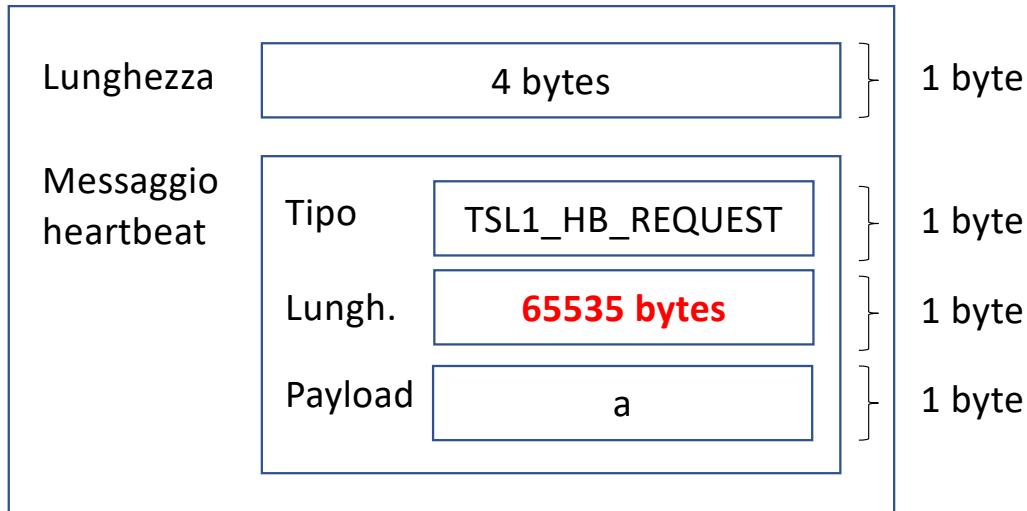


```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```

```
/* Read type and payload length first */  
hbtype = *p++;  
n2s(p, length);  
pl = p;
```

```
/* Enter response type, length and copy payload */  
*bp++ = TLS1_HB_RESPONSE;  
s2n(length, bp);  
memcpy(bp, pl, length);
```

## Record SSLv3

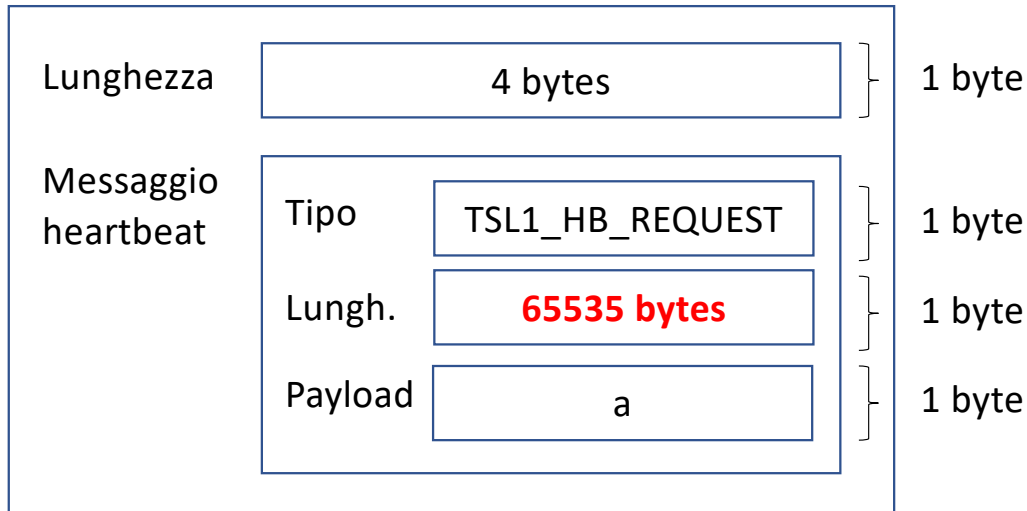


```
struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
```

```
/* Read type and payload length first */
hbtype = *p++; /* Copy type in hbtype */
n2s(p, length);
pl = p;
```

```
/* Enter response type, length and copy payload */
*bp++ = TLS1_HB_RESPONSE;
s2n(length, bp);
memcpy(bp, pl, length);
```

## Record SSLv3

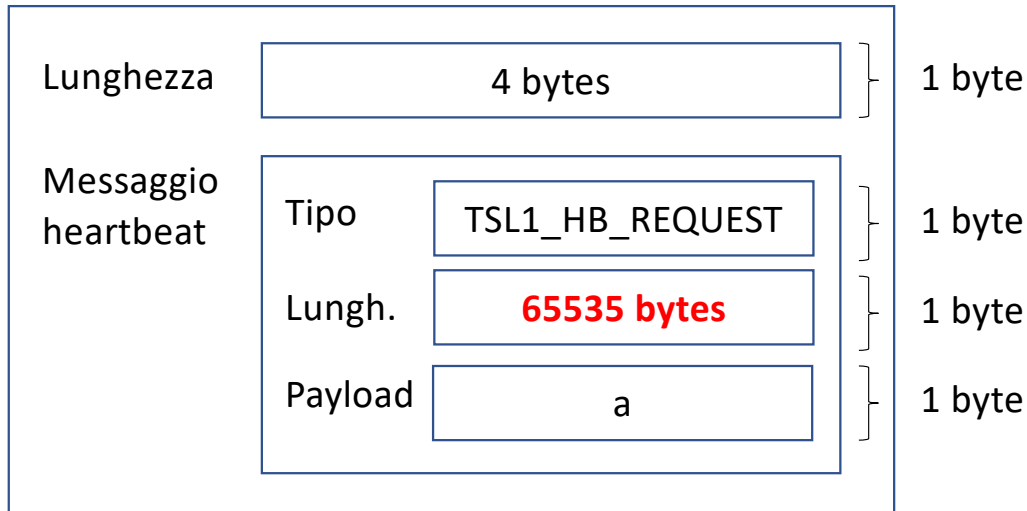


```
struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
```

```
/* Read type and payload length first */
hbtype = *p++;
n2s(p, length); /* reads wrong payload length */
pl = p;
```

```
/* Enter response type, length and copy payload */
*bp++ = TLS1_HB_RESPONSE;
s2n(length, bp);
memcpy(bp, pl, length);
```

## Record SSLv3



```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```

```
/* Read type and payload length first */
```

```
hbtype = *p++;
```

```
n2s(p, length);
```

```
pl = p; /* pl points to start of payload */
```

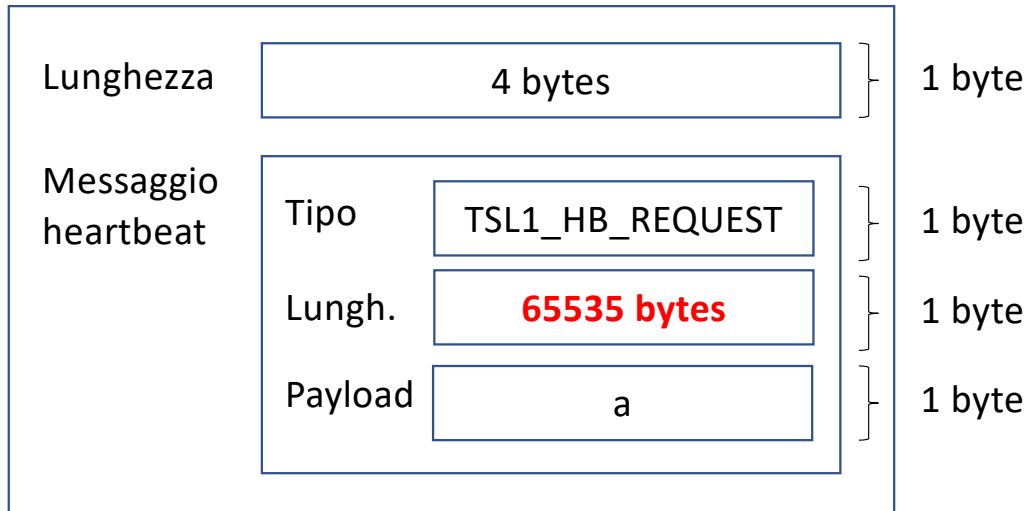
```
/* Enter response type, length and copy payload */
```

```
*bp++ = TLS1_HB_RESPONSE;
```

```
s2n(length, bp);
```

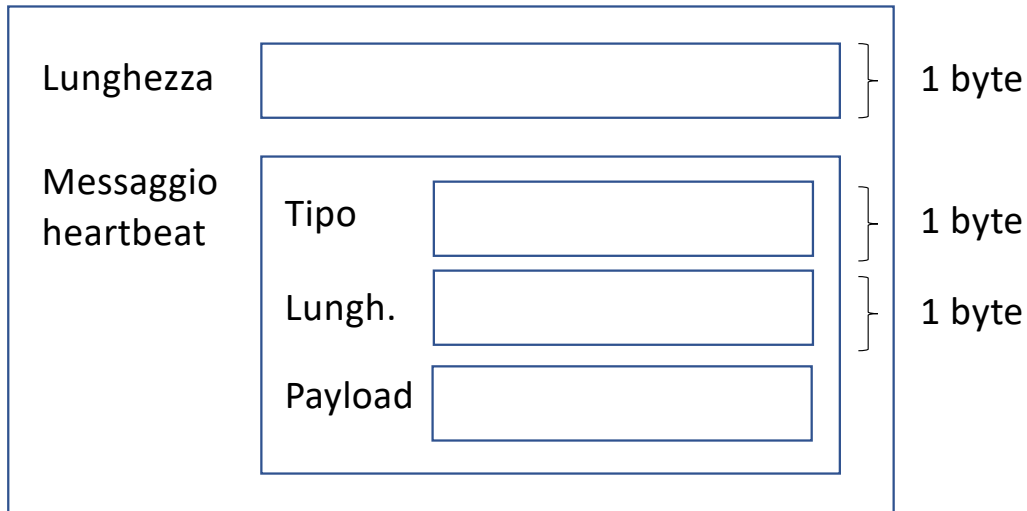
```
memcpy(bp, pl, length);
```

## Record SSLv3



```
struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
```

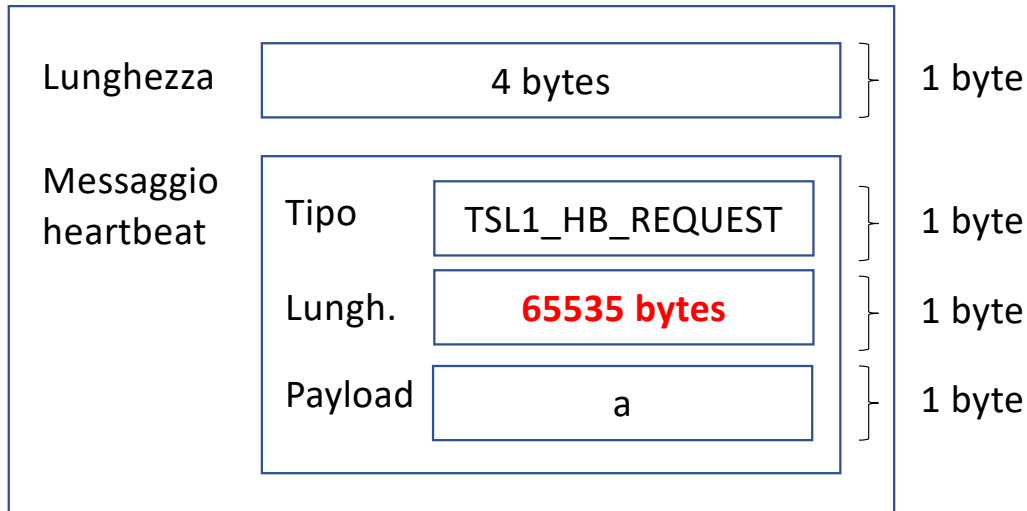
## Record SSLv3



```
/* Read type and payload length first */
hbtype = *p++;
n2s(p, length);
pl = p;
```

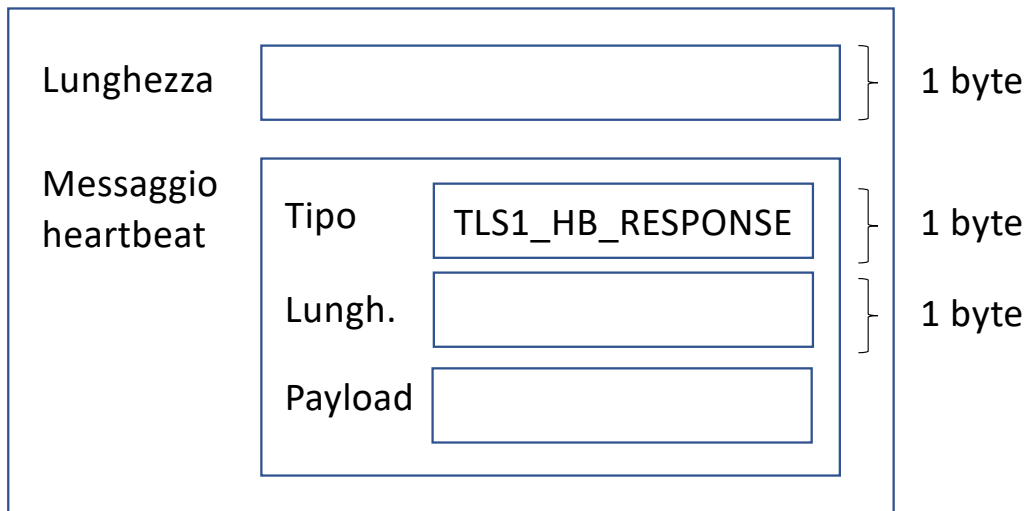
```
/* Enter response type, length and copy payload */
*bp++ = TLS1_HB_RESPONSE; /* put type */
s2n(length, bp);
memcpy(bp, pl, length);
```

### Record SSLv3



```
struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
```

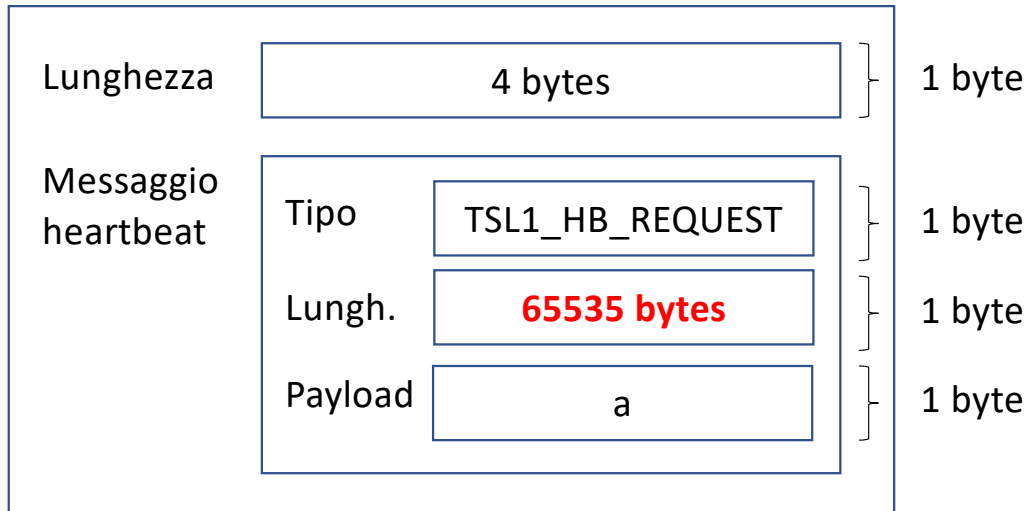
### Record SSLv3



```
/* Read type and payload length first */
hbtype = *p++;
n2s(p, length);
pl = p;
```

```
/* Enter response type, length and copy payload */
*bp++ = TLS1_HB_RESPONSE;
s2n(length, bp); /* put length */
memcpy(bp, pl, length);
```

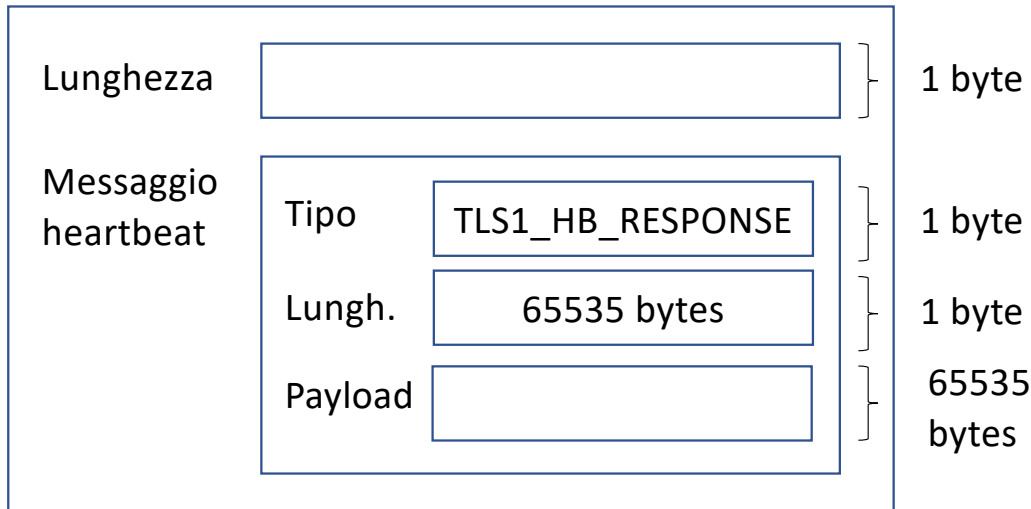
## Record SSLv3



```

struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
  
```

## Record SSLv3



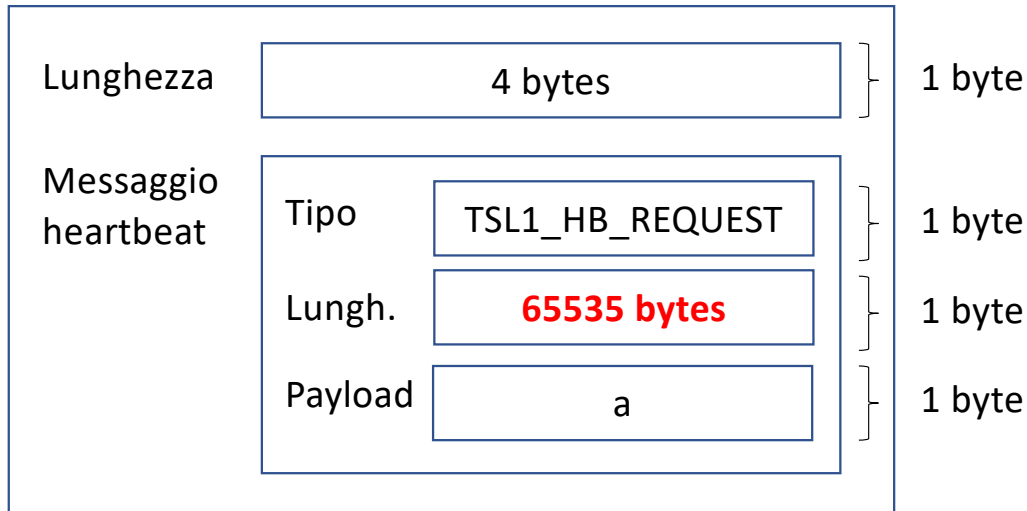
```

/* Read type and payload length first */
hbtype = *p++;
n2s(p, length);
pl = p;
  
```

```

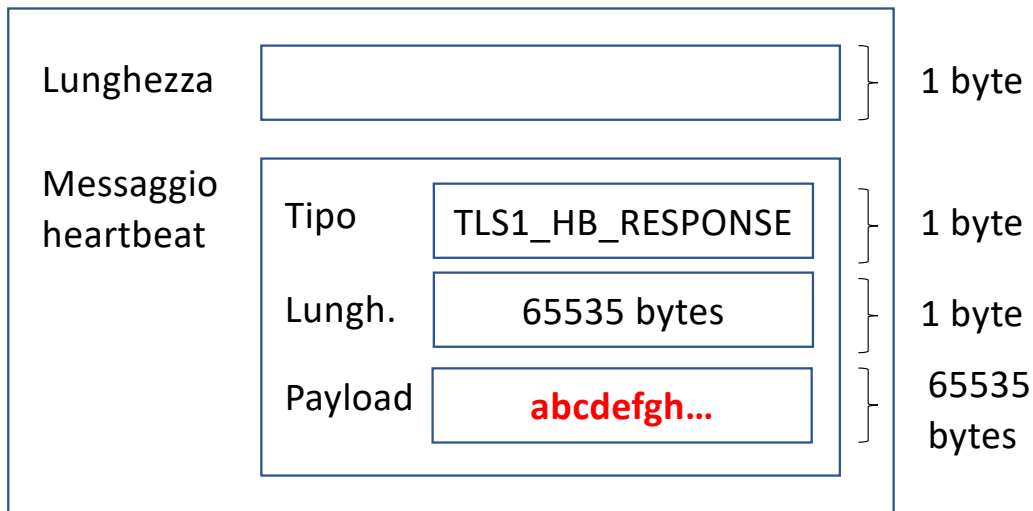
/* Enter response type, length and copy payload */
*bp++ = TLS1_HB_RESPONSE;
s2n(length, bp);
memcpy(bp, pl, length); /* memcpy length bytes! */
  
```

## Record SSLv3



```
struct {
    HeartbeatMessageType type;
    uint16 payload_length;
    opaque payload[HeartbeatMessage.payload_length];
    opaque padding[padding_length];
} HeartbeatMessage;
```

## Record SSLv3



```
/* Read type and payload length first */
hbtype = *p++;
n2s(p, length);
pl = p;
```

```
/* Enter response type, length and copy payload */
*bp++ = TLS1_HB_RESPONSE;
s2n(length, bp);
memcpy(bp, pl, length);
```

# Riferimenti aggiuntivi

- <https://heartbleed.com/>

Schemi d'attacco basati su null  
pointer dereference

## Secondo la tassonomia...

- CWE-703: Improper check or handling of exceptional conditions
  - CWE-754: Improper check for unusual or exceptional condition
    - CWE-476: NULL pointer dereference
- CWE-710: Improper adherence to coding standards
  - CWE-476: NULL pointer dereference
- CAPEC-255: Manipulate data structures
  - CAPEC-129: Pointer manipulation

# I puntatori

- Un puntatore è una variabile che contiene un indirizzo di memoria, in genere di un'altra variabile
- In C dichiariamo non solo che una variabile è di «tipo puntatore», ma anche a che tipo di dati punta l'indirizzo che contiene

|            |            |
|------------|------------|
| 0xbfb23aac | 9          |
| 0xbfb23aa8 | 0xbfb23aac |

Variable di tipo «int»  
↓  
int a = 9;  
int \*ptr = &a;  
↑  
Variable di tipo «puntatore a int»

# Usi dei puntatori

```
int main() {  
    int a = 9;  
    int *ptr = &a;  
  
    printf("Valore di a : %d\n", a );           //Stampa 9  
    printf("Indirizzo di a : %x\n", &a );       //Stampa bfb23aac  
    printf("Valore di ptr: %x\n", ptr);         //Stampa bfb23aac  
    printf("Indirizzo di ptr : %x\n", &ptr );   //Stampa bfb23aa8  
    printf("Valore di *ptr : %d\n", *ptr);      //Stampa 9  
  
    return 0;  
}
```

|            |            |
|------------|------------|
| 0xbfb23aac | 9          |
| 0xbfb23aa8 | 0xbfb23aac |

# Il puntatore NULL

- L'indirizzo di memoria 0 viene anche chiamato NULL
- Per l'hardware è un indirizzo di memoria come un altro, ma il sistema operativo rende la memoria all'indirizzo 0 non utilizzabile dalle applicazioni: ogni tentativo di utilizzare (dereferenziare) NULL per accedere alla memoria genera un segmentation fault
- Per il linguaggio di programmazione C, NULL è il valore che il programmatore assegna alle variabili puntatore quando tali variabili non contengono un indirizzo di memoria valido (non puntano a niente)

# Esempio di uso di NULL

```
int main() {  
    int a = 9;  
    int *ptr = NULL;  
  
    printf("Valore di a : %d\n", a );           //Stampa 9  
    printf("Indirizzo di a : %x\n", &a );       //Stampa bfb23aac  
    printf("Valore di ptr: %x\n", ptr);         //Stampa 0  
    printf("Indirizzo di ptr : %x\n", &ptr );   //Stampa bfb23aa8  
    printf("Valore di *ptr : %d\n", *ptr);      //Segmentation fault  
  
    return 0;  
}
```

|            |   |
|------------|---|
| 0xbfb23aac | 9 |
| 0xbfb23aa8 | 0 |

# Memoria e processi (1)

- Il sistema operativo assegna ad ogni processo un proprio spazio di indirizzamento virtuale sul quale il processo opera
- Il sistema operativo, tramite la MMU, mappa (alla granularità della pagina) gli indirizzi virtuali sugli indirizzi fisici della memoria centrale
  - Es. l'indirizzo di memoria virtuale 0xbfb23aac del processo Firefox può corrispondere all'indirizzo fisico 0xaac9fbac
  - Questa cella di memoria fisica è differente da quella che corrisponde allo stesso indirizzo di memoria virtuale 0xbfb23aac del processo Thunderbird (ad esempio la cella di memoria di indirizzo fisico 0x427281ac)

## Memoria e processi (2)

- Non tutto lo spazio di indirizzamento virtuale di un processo (che può essere enorme) è mappato su memoria fisica
- Un processo può accedere solo alla parte del proprio spazio di indirizzamento virtuale mappato su memoria fisica
- Se il processo prova ad accedere ad un indirizzo virtuale non mappato su un indirizzo fisico, viene generato un segmentation fault e il processo viene terminato
- In particolare la prima pagina di ogni processo (che contiene l'indirizzo virtuale 0, corrispondente a NULL) non viene mai mappata su una pagina fisica: per tale motivo un accesso alla memoria con il puntatore NULL genera un segmentation fault

# Assegnare la pagina 0

- I sistemi operativi moderni però mettono a disposizione chiamate di sistema attraverso le quali i processi possono chiedere di mappare pagine di memoria virtuale su pagine di memoria fisica
- Nei sistemi UNIX e UNIX-like tale chiamata è mmap
- Usando tali chiamate è possibile mappare la pagina 0 all'interno dello spazio di indirizzi virtuali di un processo su una pagina di memoria fisica, rendendo l'indirizzo 0 (NULL) un indirizzo valido
- Pertanto non avremo più segmentation fault quando dereferenziamo NULL!

# L'API mmap

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

- Mappa degli indirizzi fisici (o un file) nello spazio di memoria virtuale del processo che la invoca
- Parametri
  - *addr* l'indirizzo virtuale a partire dal quale si vuole mappare la memoria fisica
  - *length* la lunghezza della regione di memoria da mappare. La memoria si può mappare solo per multipli della dimensione di una pagina di memoria, cioè 4k (4096 bytes) su processori x86.
  - *prot* indica quali permessi ha quella regione di memoria , può assumere i valori PROT\_READ, PROT\_WRITE, PROT\_EXEC, or PROT\_NONE
  - *flags* opzioni sul mapping. MAP\_FIXED ad esempio indica che la memoria deve essere proprio allocata nell'indirizzo passato dal parametro *addr* oppure la chiamata deve fallire
  - *fd* viene usato per mappare un file anzichè della memoria fisica; serve per poter fare accessi diretti e rapidi al contenuto del file. Se non si intende usare MMAP per mappare un file occorre passare MAP\_ANONYMOUS tra i flags
  - *offset* viene usato in combinazione con *fd* per indicare la porzione del file che intendiamo mappare
  - *mmap* restituisce l'indirizzo della memoria mappata, oppure la costante MAP\_FAILED se la chiamata fallisce

# Esempio

```
int main() {
    int *ptr = NULL; //dichiaro un puntatore ad intero;

    mmap(0, 4096, PROT_READ|PROT_WRITE,
        MAP_PRIVATE|MAP_ANONYMOUS|MAP_FIXED, -1, 0)

    printf("Valore di ptr: %x\n", ptr);    //Stampa 0
    printf("Valore di *ptr : %d\n", *ptr); //Stampa un valore "sporco" ma non fallisce
    printf("\n\nAssegno 17 a *ptr\n\n\n");
    *ptr = 17; //Sebbene ptr sia NULL il dereferenzamento va a buon fine: dal momento che
              //la pagina di memoria virtuale 0 è stata mappata su una pagina fisica,
              //l'istruzione *ptr = 17 scrive il valore 17 nelle celle di memoria di indirizzo
              //0, 1, 2 e 3

    printf("Valore di ptr: %x\n", ptr);    //Stampa 0
    printf("Valore di *ptr: %d\n", *ptr); //Stampa 17

    return 0;
}
```

# Dove trovare gli esempi

- Gli esempi proposti nelle prime slides si trovano sulla macchina virtuale del corso in `Workspaces/workspaceNullDereference/Tutorial`
- Per poterli eseguire bisogna utilizzare i seguenti comandi:

```
$ su (password: root)
```

```
$ echo 0 > /proc/sys/vm/mmap_min_addr
```

```
$ gcc Ptr1_NullPointerDeref.c
```

```
$ ./a.out
```

```
$ gcc Ptr1_NullPointerDeref_MMAP.c
```

```
$ ./a.out
```

# Sfruttare null pointer dereference

- Supponiamo che un programma ad elevati privilegi effettui erroneamente, per particolari combinazioni di input, una null pointer dereference
- Grazie alla possibilità di mappare la pagina 0 su una pagina fisica è possibile sfruttare questa vulnerabilità per iniettare codice e dati malevoli nell'applicazione e fare in modo che questa li usi / esegua
- Noi considereremo il caso in cui il programma vulnerabile sia il kernel del sistema operativo

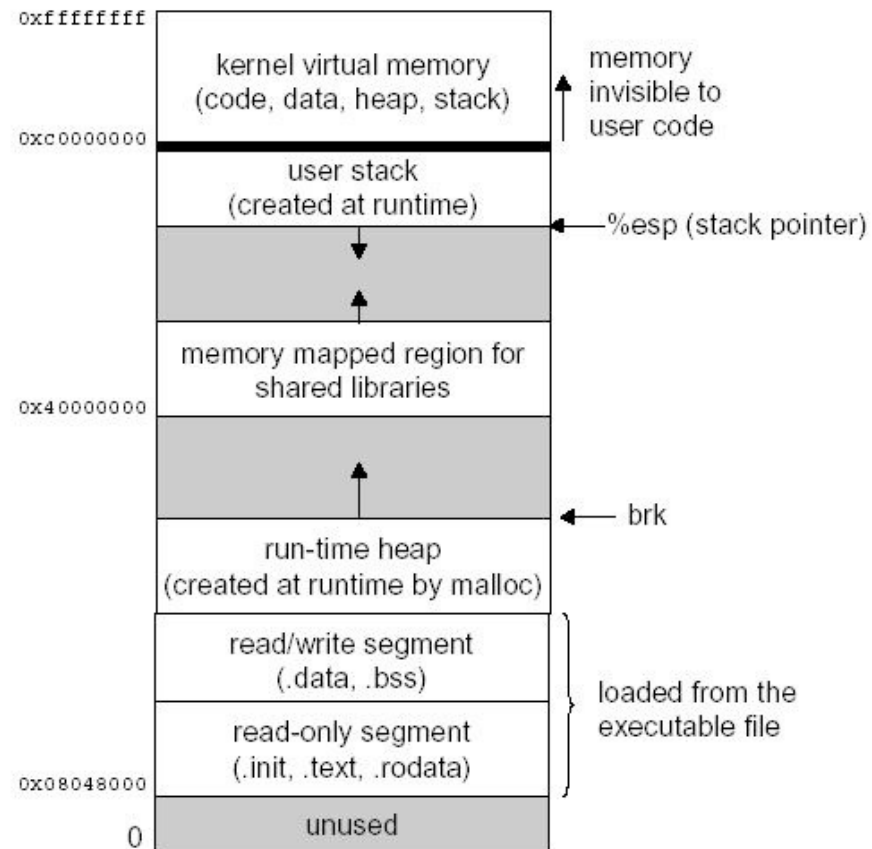
# Anche il kernel può contenere difetti

- Una null pointer dereference nel codice del kernel può essere potenzialmente sfruttata per iniettare dati o codice malevoli all'interno del kernel
- Spesso il modulo del sistema operativo che presenta una null pointer dereference è un device driver (i driver sono i tipici “punti deboli” dei sistemi operativi)

# Il kernel mappato in memoria

- Per rendere meno onerosa l'invocazione di una system call, il codice del kernel viene mappato in una regione dello spazio di indirizzamento virtuale di ogni processo
- Tale regione viene protetta dalle interferenze del processo stesso:
  - La memoria dove è mappato il kernel è accessibile solo quando la CPU è in modalità privilegiata (contesto kernel)
  - I processi possono entrare in modalità privilegiata solo invocando alcuni entry point del kernel ben definiti (le system call)
  - Al ritorno da una system call il processo esce dalla modalità privilegiata e ritorna nel proprio contesto originario
- In modalità privilegiata, il kernel sfrutta la MMU per «vedere» il VAS del processo nel cui contesto è stato invocato
- Ma questo implica che l'indirizzo 0 usato dal kernel **non** è l'indirizzo fisico 0: è l'indirizzo logico 0 del processo utente!

# Mappa della memoria di un processo (Linux)



# NULL pointer dereference di puntatori a funzione

- Particolarmente grave è quando il puntatore NULL che viene dereferenziato è un puntatore a funzione
  - Un puntatore a funzione è un indirizzo di memoria che punta ad una funzione da eseguire
  - Permette di definire algoritmi generici che applicano funzioni specifiche ai dati
  - Tipicamente usati nei driver di periferica
- Se mappiamo l'indirizzo 0 all'interno dello spazio di indirizzi del processo possiamo far sì che l'uso di un puntatore a funzione posto a NULL generi l'esecuzione da parte del kernel di una funzione da noi desiderata

# Schema d'attacco basato su NULL pointer dereference

- Il processo malevolo esegue mmap, mappando la pagina virtuale 0 su una pagina fisica
- Quindi scrive a partire dall'indirizzo zero il codice malevolo
- Infine esegue una chiamata di sistema configurata in maniera tale che
  - Un puntatore a funzione nel kernel sia impostato a NULL, e
  - Il kernel usi tale puntatore per invocare la funzione, eseguendo il codice malevolo residente all'indirizzo (logico) zero
- Il codice malevolo viene eseguito in contesto kernel
- Di solito il codice malevolo:
  - Assegna i permessi di root al processo malevolo di partenza (il kernel può assegnare permessi di root ad un qualsiasi processo)
  - Esegue opportuni comandi, ad esempio shellcode

# Possibili contromisure

- I kernel recenti di Linux hanno un parametro di configurazione detto `mmap_min_addr` che indica l'indirizzo virtuale minimo che si può rimappare con l'API `mmap` (questo indirizzo di default è 4096)
- Tuttavia è possibile sfruttare altre vulnerabilità per superare questo limite (si veda CVE-2009-1895)

La vulnerabilità CVE-2009-2692

# La vulnerabilità CVE-2009-2692 (1)

- Vulnerabilità di tipo NULL pointer dereference nei kernel di Linux
- Mancata verifica di NULL pointers nelle struct socket
- Una socket è un tipo speciale di file impiegato per la comunicazione di rete
- Il kernel mantiene una struct socket per ogni socket
- Un campo della struct socket punta ad una struct proto\_ops, che contiene, a seconda del tipo di socket, puntatori a funzioni che implementano diverse operazioni sulle socket stesse, come ad esempio accept, bind, shutdown
- Ognuno di questi puntatori può essere NULL se il tipo di socket considerato non implementa l'operazione corrispondente
- Vulnerabilità: la funzione sock\_sendpage non controlla che il corrispondente puntatore sia diverso da NULL

## La vulnerabilità CVE-2009-2692 (2)

```
//questa e' una funzione del kernel corretta
static ssize_t sock_splice_read(struct file *file, ....) {
    struct socket *sock = file->private_data;

    //quello che segue e' il codice che fa il controllo previsto
    if ( sock->ops->splice_read == NULL )
        return -EINVAL;

    return sock->ops->splice_read(sock, ppos, pipe, len, flags);
}
```

## La vulnerabilità CVE-2009-2692 (3)

```
//questa e' la funzione del kernel vulnerabile
static ssize_t sock_sendpage(struct file *file...) {
    struct socket *sock = file->private_data;
    ... //nessun controllo su sock->ops->sendpage...
    return sock->ops->sendpage(sock, page, offset, size, flags);
}
```

# La vulnerabilità CVE-2009-2692 (4)

- `sock_sendpage` è invocata, ad esempio, dalla system call  
`int sendfile(int fd, int sock, off_t offset, off_t *len, struct sf_hdr *hdr, int flags);`
- Kernel colpiti: tutti quelli rilasciati da maggio 2001 ad agosto 2009
  - Linux 2.4, da 2.4.4 sino 2.4.37.4 (incluso)
  - Linux 2.6, da 2.6.0 sino a 2.6.30.4 (incluso)
- Questi kernel non implementavano il parametro di configurazione `mmap_min_addr`

# Esempio di exploit

- La VM del corso contiene un programma che implementa un exploit
- L'exploit esegue mmap, crea una socket ed invoca delle funzioni sulla socket in maniera opportuna per causare un NULL pointer dereference
- Il programma è implementato dal file  
Workspaces/workspaceNullDereference/CVE-2009-2692/exploit.final.c
- Per eseguirlo è necessario riavviare la VM e selezionare il kernel con il nome "Buggy", e quindi eseguire da shell

```
$ cd Workspaces/workspaceNullDereference/CVE-2009-2692/exploit  
$ gcc -o exploit exploit.final.c  
$ ./exploit
```

- A questo punto potrete osservare una shell di root

La vulnerabilità CVE-2009-1897

# La vulnerabilità CVE-2009-1897 (1)

- Questa vulnerabilità è una NULL pointer dereference di un puntatore **non** a funzione: ciononostante è sfruttabile per ottenere l'esecuzione di codice malevolo
- La vulnerabilità è nel driver TUN/TAP (packet tunneling) di alcune versioni di Linux
- Il codice del driver accede (funzione `tun_chr_poll`) ad una struct sock, simile alla struct socket che abbiamo visto in precedenza

## La vulnerabilità CVE-2009-1897 (2)

```
static unsigned int tun_chr_poll(struct file *file, ....) {  
    struct tun_file *tfile = file->private_data;  
    struct tun_struct *tun = __tun_get(tfile)  
    struct sock *sk = tun->sk;  
  
    //controllo su tun - troppo tardi!  
    if (!tun)  
        return POLLERR;  
  
    if (... !test_and_set_bit(SOCK_ASYNC_NOSPACE,  
        &sk->sk_socket->flags) ...)  
}
```

# L'exploit (1)

- GCC rileva che il null check su tun è fatto dopo il suo uso, e lo elimina (cosa permessa dallo standard C)
- L'attaccante rimappa NULL su una pagina sotto il suo controllo, e così facendo può controllare il contenuto di tun, e di `sk == tun->sk`
- Quindi imposta il contenuto di `sk->sk_socket.flags` con l'indirizzo di uno dei puntatori a funzione nella struct `proto_ops` del driver TUN/TAP
- In particolare, `sk->sk_socket.flags` è impostato con l'indirizzo del puntatore alla funzione `mmap()`, che il driver aveva impostato a NULL in fase di inizializzazione (TUN/TAP non supporta `mmap()`)

## L'exploit (2)

- La `test_and_set_bit` pertanto imposta a 1 il bit meno significativo (`SOCK_ASYNC_NOSPACE`) di `sk->sk_socket->flags`
- In sintesi, il puntatore alla funzione `mmap()` del driver TUN/TAP ora ha valore `0x1`, che è ancora un indirizzo nella pagina sotto controllo dell'attaccante: L'attaccante può porre quindi lo shellcode in memoria a partire dall'indirizzo `0x1`
- A questo punto l'attaccante invoca la funzione `mmap()` del driver TUN/TAP: dal momento che `0x1` non è NULL viene considerato valido dal driver e viene quindi eseguito lo shellcode in modalità kernel

# Riferimenti aggiuntivi

- Vulnerabilità del kernel linux CVE-2009-2692
  - <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2009-2692>
- Vulnerabilità del kernel linux CVE-2009-1897
  - <https://lwn.net/Articles/342330/>
  - <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2009-1897>

# Java secure coding guidelines

# Java secure coding guidelines

- Presentiamo alcune linee guida per lo sviluppo di codice Java sicuro
- Esse sono raggruppabili in due categorie, domain dependent e domain independent
- Domain dependent guidelines:
  - Linee guida per cui è difficile definire dei controlli automatici
- Domain independent guidelines
  - Linee guida che si riferiscono alla struttura del codice per cui è automatizzabile il controllo
  - SpotBugs e SonarQube sono due esempi di tools che effettuano anche questo genere di controlli
  - FindBugs (una versione precedente di SpotBugs) è installato nella VM del corso come eclipse plugin, usate il workspace Workspaces/workspaceCodeGuidelines per osservare cosa viene trovato da FindBugs
- Riferimento: Long et al., Java Coding Guidelines: 75 Recommendations for Reliable and Secure Programs
- Elenco completo e aggiornato di security guidelines:  
<https://wiki.sei.cmu.edu/confluence/display/java/SEI+CERT+Oracle+Coding+Standard+for+Java>

# Limitare la durata della vita dei dati sensibili

- Dati sensibili potrebbero essere ispezionati da un attaccante se l'applicazione
  - Usa oggetti per memorizzare dati sensibili, e gli oggetti non sono ripuliti/garbage collected
  - Ha pagine di memoria che vengono scaricate (swap) su disco
  - Mostra i dati sensibili in messaggi di debugging
- Soluzione: ripulire i dati
  - Es. sovrascrivere la stringa usata per ricevere la pwd dell'utente
  - In C tale codice potrebbe essere eliminato dalle ottimizzazioni

# Limitare la durata della vita dei dati sensibili

NO

```
public static void main(String args[]){  
    int i;  
    for( i = 0; i < args.length; i++ ){  
        /* Do something*/  
    }  
}
```

SI

```
public static void main(String args[]){  
    for( int i = 0; i < args.length; i++ ){  
        /* Do something*/  
    }  
}
```

# Memorizzare gli hash delle password anziché le password stesse

- Password salvate in plain text potrebbero essere osservate da un attaccante
- Ma anche password crittografate sono rischiose (devo pur salvare da qualche parte la chiave...)
- Utilizzare hash functions ha vantaggi:
  - Poco costose
  - L'inverso è computationally infeasible
- Evitare MD5, preferire SHA256

# Rendere le classi non serializzabili (se non è necessario)

- La serializzazione in alcuni contesti potrebbe essere pericolosa
- Questo perchè permette ad un attaccante di accedere allo stato interno degli oggetti

```
private final void writeObject(ObjectOutputStream out)
throws java.io.IOException {
    throw new java.io.IOException("Object cannot be serialized");
}
```

# Rendere le classi non serializzabili anche non deserializzabili

- Un attaccante potrebbe creare un oggetto ad hoc che può essere deserializzato anche se la classe non è serializzabile!

```
private final void readObject(ObjectInputStream in)
throws java.io.IOException {
    throw new java.io.IOException("Class cannot be deserialized");
}
```

# Rendere le classi non clonabili (se non è necessario)

- Un attaccante potrebbe evitare di eseguire il costruttore della vostra classe (che potrebbe contenere security checks)
- Ad esempio può creare una sottoclasse che si limita ad implementare il metodo “clone”
- In tal modo potrebbe produrre stati che non sarebbe autorizzato a produrre

```
public final void clone() throws java.lang.CloneNotSupportedException {  
    throw new java.lang.CloneNotSupportedException();  
}
```

# Gestire integer overflow

```
public static void main(String[] args) {  
    int x = Integer.MAX_VALUE-2;  
    for ( int i = 0; i < 4; i++ ){  
        x=x+1;  
        System.out.println(x);  
    }  
}
```

Il programma avrà il seguente output:

2147483646

2147483647

-2147483648 (a questo punto abbiamo avuto un integer overflow)

-2147483647

# Gestire integer overflow con Java 8

```
public static void main(String[] args) {  
    int x = Integer.MAX_VALUE-2;  
    for ( int i = 0; i < 4; i++ ){  
        x=Math.addExact(x,1);  
        System.out.println(x);  
    }  
}
```

Il programma avrà il seguente output:

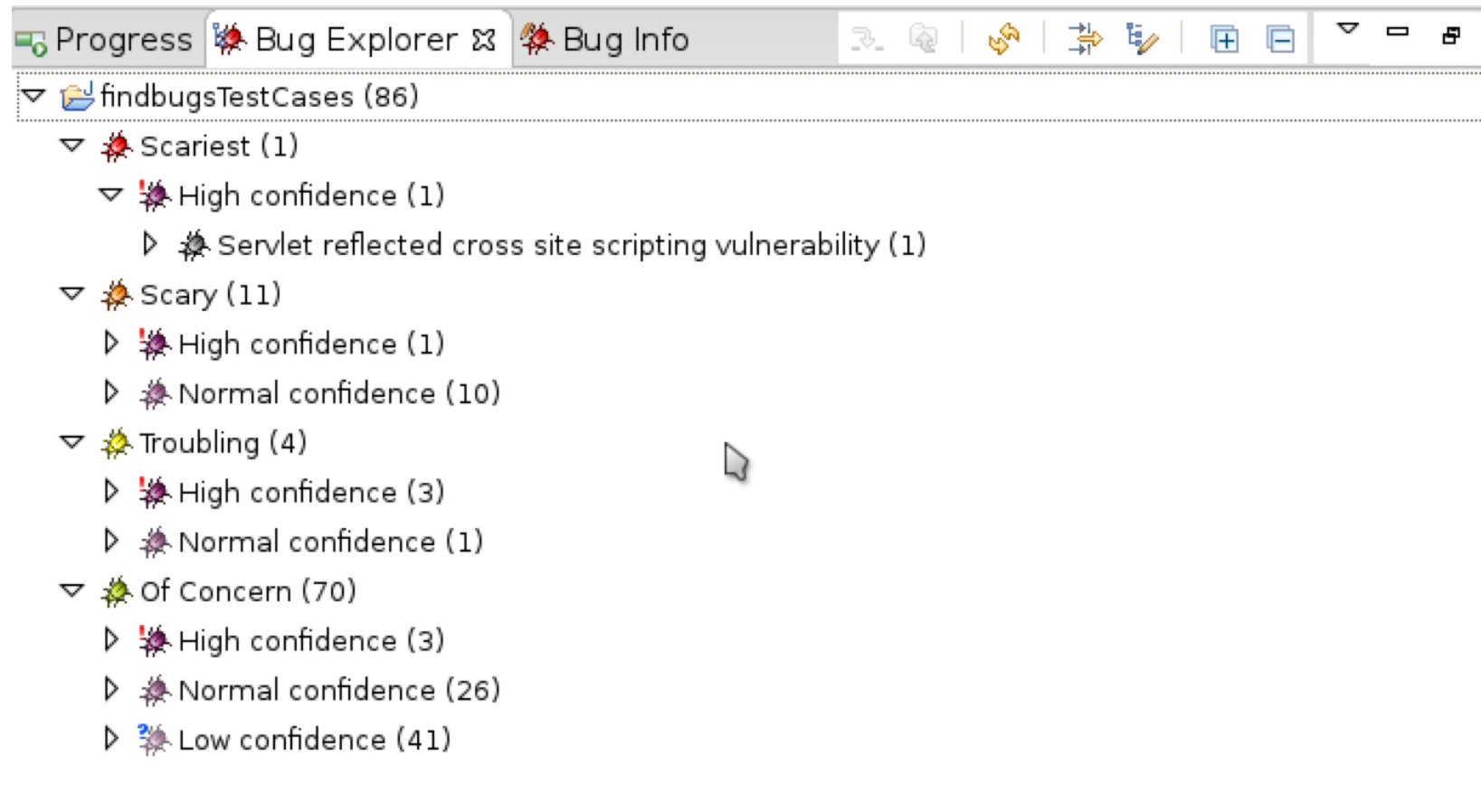
2147483646

2147483647

ArithmeticException-...

# SpotBugs domain independent guidelines

- Di seguito vengono riportate alcune linee guida domain independent per cui il controllo è automatizzabile.
- Per esporle viene fatto riferimento al codice di errore riportato da SpotBugs
  - Strumento di analisi statica
  - Console + GUI + *Eclipse Plug-in*



# May expose internal representation by returning reference to mutable object

- SpotBugs: **EI\_EXPOSE\_REP (EI)**
- Returning a reference to a mutable object value stored in one of the object's fields exposes the internal representation of the object.
- If instances are accessed by untrusted code, and unchecked changes to the mutable object would compromise security or other important properties, you will need to do something different. Returning a new copy of the object is better approach in many situations.

```
public class Bug1726846 {
    private static final int[] INTS = { 1, 2, 3 };
    public static final int[] getInts() {
        return INTS;
    }
}
```

- getInts non segue questa linea guida: restituisce il riferimento all'array, e il chiamante può modificare il contenuto di INTS
- final indica che la variabile INTS non può essere modificata, tuttavia il contenuto dell'array può essere modificato.
- Necessario restituire una copia di INTS
  - return System.arraycopy( src, 0, new int[INTS.length], 0, src.length );
  - per le collections è bene usare Collections.unmodifiableXXXXX
- Altra possibilità: usare metodi delegate:

```
private static final int getInt( int position ){
    return INTS[position];
}
```

- Se però la collezione contiene, a sua volta, oggetti mutabili, copiare la collezione non è sufficiente...

# May expose internal representation by incorporating reference to mutable object

- SpotBugs: **EI\_EXPOSE\_REP2 (EI2)**
- This code stores a reference to an externally mutable object into the internal representation of the object.
- If instances are accessed by untrusted code, and unchecked changes to the mutable object would compromise security or other important properties, you will need to do something different. Storing a copy of the object is better approach in many situations.

```
public void setSqlDate(java.sql.Date sqlDate) {  
    this.sqlDate = sqlDate;  
}
```

- Il chiamante potrebbe modificare il contenuto di SqlDate se mantiene un riferimento ad esso.
- Meglio creare una copia di sqlDate prima di assegnarla a this.sqlDate.
- Notare che è meglio non invocare clone() per creare una copia della data: clone potrebbe effettuare una shallow copy
- Se esiste un copy constructor, è meglio usare quello

```
public void setSqlDate(java.sql.Date sqlDate) {  
    this.sqlDate = new java.sql.Date(sqlDate.getTime());  
}
```

# Empty database password

- SpotBugs: **DMI\_EMPTY\_DB\_PASSWORD**
- This code creates a database connect using a blank or empty password.
- This indicates that the database is not protected by a password.

```
String driverName = "oracle.jdbc.driver.OracleDriver";  
Class.forName(driverName);  
String url = "";  
String username = "";  
String password = "";  
  
// Bug: We only close the resources opened in the last iteration  
for (int i = 0; i < 10; i++) {  
    connection = DriverManager.getConnection(url, username,  
        password);  
}
```

# Hardcoded constant database password

- SpotBugs: **DMI\_CONSTANT\_DB\_PASSWORD**
- This code creates a database connect using a hardcoded, constant password.
- Anyone with access to either the source code or the compiled code can easily learn the password, and use it to access the DB with a DBMC client.

```
Class.forName("com.mysql.jdbc.Driver").newInstance();  
Connection conn = DriverManager.getConnection(  
    "jdbc:mysql://localhost/"  
    + "database", "username", "password");
```

Field isn't final but should be

Field isn't final but should be refactored to be so

- SpotBugs: **MS\_SHOULD\_BE\_FINAL / MS\_SHOULD\_BE\_REFACTORED\_TO\_BE\_FINAL**
- This static field is public but not final, and could be changed by malicious code or by accident from another package
- The field could be made final to avoid this vulnerability